Nutan Maharashtra Vidya Prasarak Mandal's

NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Computer Engineering

LABORATORY MANUAL

SUBJECT: Data Structures Laboratory

[SUBJECT CODE: 210247]

CLASS: S.E

YEAR: 2023-24

PREPARED BY: **APPROVED BY:**

(Prof.S.V.Reddy) **H.O.D. [Computer]**

Vision and Mission of the Institute

1. **Vision of the Institute-**To be a notable institution for providing quality technical education, ensuring ethical, moral, and holistic development of students.
2. **Mission of the Institute**- To nurture engineering graduates with highest technical competence, professionalism and problem solving skills to serve the needs of industry and society.

Vision and Mission of the Department

Department Vision:

Imbibing Quality Technical Education and Overall Development by Endowing Students with Societal and Ethical skills in Computer Engineers

Department Mission:

- To impart engineering knowledge and skills by adopting effective teaching learning processes..
- To develop professional, entrepreneurial & research competencies encompassing continuous intellectual growth
- To produce educated students to exhibit societal and ethical responsibilities in the working environment.

Program Outcomes

1. **Engineering knowledge**:

Graduates can apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to Civil Engineering related problems.

2. **Problem analysis**:

An ability to identify, formulate, review research literature, and analyse Civil engineering problems reaching substantiated conclusions using principles of mathematics and engineering sciences.

3. **Design/development of solutions**:

An ability to plan, analyse, design, and implement engineering problems and design system components or processes to meet the specified needs.

4. **Conduct investigations of complex problems**:

An ability to use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. **Modern tool usage**:

An ability to apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society**:

An ability to apply contextual knowledge to assess societal, legal issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability**:

An ability to understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. **Ethics**:

An ability to apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. **Individual and teamwork**:

An ability to function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings to accomplish a common goal.

10. **Communication**:

An ability to communicate effectively on engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation and make effective presentations.

11. **Project management and finance**:

Ability to demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**:

An ability to engage in independent and life-long learning in the broadest context of technological change.

Program Educational Objectives (PEOs)

PEO 1: To provide core theoretical and practical knowledge in the domain of Computer Science & Engineering for leading successful career in industries, pursuing higher studies or entrepreneurial endeavours.

PEO 2: To develop the ability to critically think, analyze and make decisions for offering techno-commercially feasible and socially acceptable solutions to real life problems in the areas of computing.

PEO 3: To imbibe lifelong learning, professional and ethical attitude for embracing global challenges and make positive impact on environment and society.

Program Specific Outcomes (PSOs)

PSO 1: Able to identify suitable data structures and algorithms to design, develop and evaluate effective solutions for real-life and research problems.

PSO 2: Able to excel in various programming/project competitions and technological challenges laid by professional societies.

Course Outcomes (CO)

Course Outcomes		Level
CO1	Discover appropriate data structure that provide necessary functionalities meeting a given set of user requirements	L4
CO2	Develop a program to sort a list of records based on a given key in the record.	L3
CO3	Identify the space complexity and time complexity of various data structure operations.	L3
CO4	Make use of efficient data structure to represent given data.	L3
CO5	Simulate a program to convert an arithmetic expression from one notation to another.	L3
CO6	Simulate memory allocation and garbage collection using linked lists.	L3
CO7	Utilize one's ability as an individual for development of effective communication, practical skill and document design.	L3

List of Experiments with Mapping

DSL LAB Assignments AY-2023-24

Sr. No.	Group	Title	CO	PO
1	A	In second year computer engineering class, group A student's play cricket, group B students play badminton and group C students play football. Write a P**ython** program using functions to compute following: - a) List of students who play both cricket and badminton b) List of students who play either cricket or badminton but not both c) Number of students who play neither cricket nor badminton d) Number of students who play cricket and football but not badminton. (Note- While realizing the group, duplicate entries should be avoided, Do not use SET built-in functions)	1	1,2,3,4
2	A	Write a P**ython** program to compute following operations on String: a) To display word with the longest length b) To determines the frequency of occurrence of particular character in the string c) To check whether given string is palindrome or not d) To display index of first appearance of the substring e) To count the occurrences of each word in a given string	1	1,2,3,4
3	A	Write a **Python** program to compute following computation on matrix: a) Addition of two matrices B) Subtraction of two matrices c) Multiplication of two matrices d) Transpose of a matrix	1	1,2,3,4
4	B	a) Write a **python**program to store roll numbers of student in array who attended training program in random order. Write function for searching whether particular student attended training program or not, using Linear search and Sentinel search. b) Write a **python** program to store roll numbers of student array who attended training program in sorted order. Write function for searching whether particular student attended training program or not, using Binary search and Fibonacci search	2	1,2,3,4
5	B	Write a **python**program to store first year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using a) Selection Sort	2	1,2,3,4

		b) Bubble sort and display top five scores.		
6	**B**	Write a **python** program to store second year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using a) Insertion sort b) Shell Sort and display top five scores	**2**	**1,2,3,4**
7	**C**	Department of Computer Engineering has student's club named 'Pinnacle Club'. Students of second, third and final year of department can be granted membership on request. Similarly one may cancel the membership of club. First node is reserved for president of club and last node is reserved for secretary of club. Write C++ program to maintain club member‘s information using singly linked list. Store student PRN and Name. Write functions to: a) Add and delete the members as well as president or even secretary. b) Compute total number of members of club c) Display members d) Two linked lists exists for two divisions. Concatenate two lists.	**3**	**2,3,4**
8	**C**	Write C++ program for storing binary number using doubly linked lists. Write functions- a) To compute 1‘s and 2‘s complement b) Add two binary numbers	**3**	**2,3,4**
9	**D**	A palindrome is a string of character that‘s the same forward and backward. Typically, punctuation, capitalization, and spaces are ignored. For example, “Poor Dan is in a droop” is a palindrome, as can be seen by examining the characters “poor danisina droop” and observing that they are the same forward and backward. One way to check for a palindrome is to reverse the characters in the string and then compare with them the original-in a palindrome, the sequence will be identical. Write C++ program with functions- a) To print original string followed by reversed string using stack b) To check whether given string is palindrome or not	**1,4**	**1,2,3,4**
10	**D**	Implement C++ program for expression conversion as infix to postfix and its evaluation using stack based on given conditions: 1. Operands and operator, both must be single character. 2. Input Postfix expression must be in a desired format. 3. Only '+', '-', '*' and '/ ' operators are expected.	**1,4**	**1,2,3,4**
11	**E**	Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system. If the operating system does not use priorities, then the jobs are processed in the order they enter the system. Write C++ program for simulating job queue. Write functions to add job and delete job from queue.	**4**	**1,2,3,4**

12	**E**	Write program to implement a priority queue in C++ using an inorder list to store the items in the queue. Create a class that includes the data items (which should be template) and the priority (which should be int). The inorder list should contain these objects, with operator <= overloaded so that the items with highest priority appear at the beginning of the list (which will make it relatively easy to retrieve the highest item.)	**4**	**1,2,3,4**
13	**E**	A double-ended queue (deque) is a linear list in which additions and deletions may be made at either end. Obtain a data representation mapping a deque into a one-dimensional array. Write C++ program to simulate deque with functions to add and delete elements from either end of the deque.	**4**	**1,2,3,4**

Rubrics for Evaluation

Sr. No	Evaluation Criteria	Marks for each Criteria	Rubrics
1	Timely submission	5 or 10	Punctuality reflects the work ethics. Students should reflect that work ethics by completing the lab assignments and reports in a timely manner .
2	Journal Presentation	5 or 10	Students are expected to prepare the journal. The journal presentation of the course should be complete, clear, and understandable.
3	Performance	5 or 10	After performance, the students should have good knowledge of the experiment.
4	Understanding	5 or 10	The student should be able to explain methodology used for designing and developing the program/solution. Student should clearly understand the purpose of the assignment and its outcome.
5	Oral	5 or 10	The student should be able to answer the questions related to the lab assignments.

Group: A **Roll No:**
Assignment No: 1 **Date:**

Title:

In second year computer engineering class, group A students play cricket, group B students play badminton and group C students play football.
Write a python program using functions to compute following: -
a) List of students who play both cricket and badminton
b) List of students who play either cricket or badminton but not both
c) Number of students who play neither cricket nor badminton
d) Number of students who play cricket and football but not badminton.

Marks: **Sign:**

Group A

Assignment No. 1

Title: Implement Set Operations(Write This)

Objectives: (Write This)
To introduce basics of Sets and its operations.
To understand and implement the Array.
To understand how to pass arrays as parameters to functions

Problem Statement: (Write This)

In second year computer engineering class, group A students play cricket, group B students play badminton and group C students play football. Write a python program using functions to compute following: -

a) List of students who play both cricket and badminton
b) List of students who play either cricket or badminton but not both
c) Number of students who play neither cricket nor badminton
d) Number of students who play cricket and football but not badminton.

(Note- While realizing the group, duplicate entries should be avoided, Do not use SET built-in functions)

Theory:

1.1 The Python programming language

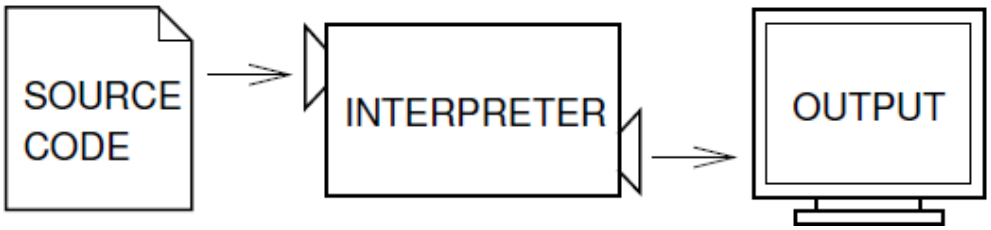
Python is an example of a high-level language; other high-level languages you might have heard of are C, C++, Perl, and Java.

As you might infer from the name "high-level language," there are also low level languages, sometimes referred to as "machine languages" or "assembly". The way of the program languages." Loosely speaking, computers can only execute programs written in low-level languages. Thus, programs written in a high-level language have to be processed before they can run. This extra processing takes some time, which is a small disadvantage of high-level languages.

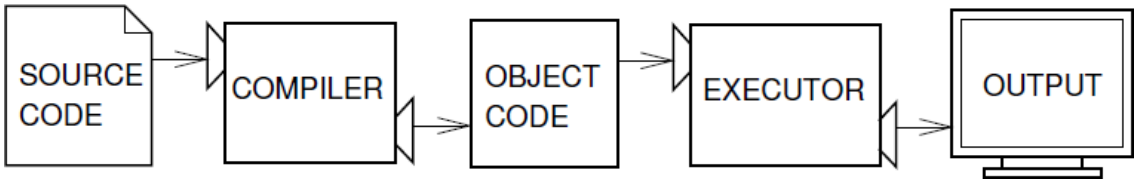
But the advantages are enormous. First, it is much easier to program in a high-level language. Programs written in a high-level language take less time to write, they are shorter and easier to read, and they are more likely to be correct. Second, high-level languages are portable, meaning that they can run on different kinds of computers with few or no modifications. Low-level programs can run on only one kind of computer and have to be rewritten to run on another. Due to these advantages, almost all programs are written in high-level languages.

Low-level languages are used only for a few specialized applications. Two kinds of programs process high-level languages into low-level languages: interpreters and compilers. An

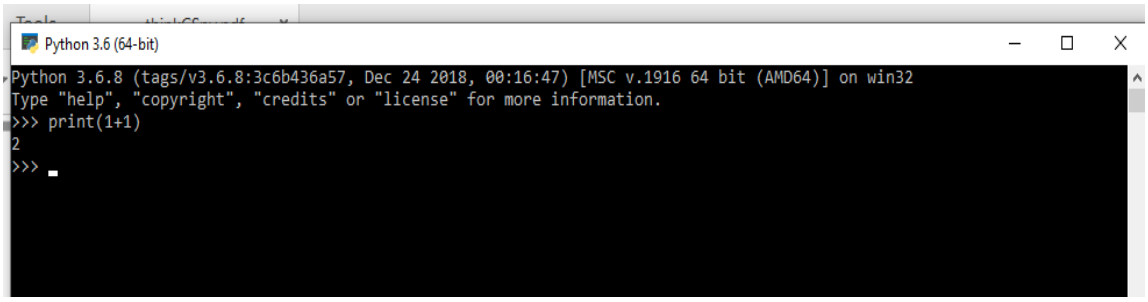
interpreter reads a high-level program and executes it, meaning that it does what the program says. It processes the program a little at a time, alternately reading lines and performing computations.



A compiler reads the program and translates it completely before the program starts running. In this case, the high-level program is called the source code, and the translated program is called the object code or the executable. Once a program is compiled, you can execute it repeatedly without further translation.



Python is considered an interpreted language because Python programs are executed by an interpreter. There are two ways to use the interpreter: commandline mode and script mode. In command-line mode, you type Python programs and the interpreter prints the result:



What is a program?

A program is a sequence of instructions that speci_es how to perform a computation. The computation might be something mathematical, such as solving a system of equations or _nding the roots of a polynomial, but it can also be a symbolic computation, such as searching and replacing text in a document or (strangely enough) compiling a program. The details look di_erent in di_erent languages, but a few basic instructions appear in just about every language:

The way of the program
input: Get data from the keyboard a file, or some other device.
output: Display data on the screen or send data to a file or other device.

math: Perform basic mathematical operations like addition and multiplication.
conditional execution: Check for certain conditions and execute the appropriate sequence of statements.
repetition: Perform some action repeatedly, usually with some variation.

What is debugging?

Programming is a complex process, and because it is done by human beings, it often leads to errors. For whimsical reasons, programming errors are called bugs and the process of tracking them down and correcting them is called debugging.
Three kinds of errors can occur in a program: syntax errors, runtime errors, and semantic errors. It is useful to distinguish between them in order to track them down more quickly.

Syntax errors

Python can only execute a program if the program is syntactically correct; otherwise, the process fails and returns an error message. Syntax refers to the structure of a program and the rules about that structure.

Runtime errors

The second type of error is a runtime error, so called because the error does not appear until you run the program. These errors are also called exceptions because they usually indicate that something exceptional (and bad) has happened.

Semantic errors

The third type of error is the semantic error. If there is a semantic error in your program, it will run successfully, in the sense that the computer will not generate any error messages, but it will not do the right thing. It will do something else. Specifically, it will do what you told it to do.

Lists

A list is an ordered set of values, where each value is identified by an index. The values that make up a list are called its elements. Lists are similar to strings, which are ordered sets of characters, except that the elements of a list can have any type. Lists and strings| and other things that behave like ordered sets | are called sequences.
There are several ways to create a new list; the simplest is to enclose the elements in square brackets ([and]):
[10, 20, 30, 40]
["spam", "bungee", "swallow"]

The first example is a list of four integers. The second is a list of three strings.
The elements of a list don't have to be the same type. The following list contains a string, a float, an integer another list:
["hello", 2.0, 5, [10, 20]]
A list within another list is said to be nested.
Lists that contain consecutive integers are common, so Python provides a simple way to create them:
>>> range(1,5)
[1, 2, 3, 4]

The range function takes two arguments and returns a list that contains all the integers from the _rst to the second, including the _rst but not including the second!
There are two other forms of range. With a single argument, it creates a list that starts at 0:
>>> range(10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Accessing elements
The syntax for accessing the elements of a list is the same as the syntax for accessing the characters of a string|the bracket operator ([]). The expressionin side the brackets specifies the index. Remember that the indices start at 0:
print numbers[0]
numbers[1] = 5

Sets:

Python‟s built-in set type has the following characteristics:

- Sets are unordered.
- Set elements are unique. Duplicate elements are not allowed.
- A set itself may be modified, but the elements contained in the set must be of an immutable type.

Example

Create a Set:

```
thisset = {"apple", "banana", "cherry"}
print(thisset)
```

{'cherry', 'banana', 'apple'}

Access Items

You cannot access items in a set by referring to an index or a key.

But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the in keyword.

Example

Loop through the set, and print the values:

```
thisset = {"apple", "banana", "cherry"}

for x in thisset:
  print(x)
```

banana
cherry
apple

Add Items

To add one item to a set use the add() method.

To add more than one item to a set use the update() method.

Example

Add an item to a set, using the add() method:

```
thisset = {"apple", "banana", "cherry"}
thisset.add("orange")
print(thisset)
```

{'orange', 'banana', 'apple', 'cherry'

Get the Length of a Set

To determine how many items a set has, use the len() method.

Example

Get the number of items in a set:

```
thisset = {"apple", "banana", "cherry"}
print(len(thisset))
```

3

Remove Item

To remove an item in a set, use the remove(), or the discard() method.

Example

Remove "banana" by using the remove() method:

```
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)
```

{„cherry“, „apple“ }

Join Two Sets

There are several ways to join two or more sets in Python.

You can use the union() method that returns a new set containing all items from both sets, or the update() method that inserts all the items from one set into another:

Example

The union() method returns a new set with all items from both sets:

```
set1 = {"a", "b" , "c"}
set2 = {1, 2, 3}

set3 = set1.union(set2)
print(set3)
```

{„c‟,‟a‟,1,3,‟b‟,2}

Sets support a standard collection of operations, including union, intersection, difference, and symmetric difference. Here‟s an example:

```
a = t | s          # Union of t and s
b = t & s          # Intersection of t and s
c = t – s          # Set difference (items in t, but not in s)
d = t ^ s          # Symmetric difference (items in t or s, but not both)
```

Set Methods

Python has a set of built-in methods that you can use on sets.

Method	Description
add()	Adds an element to the set
clear()	Removes all the elements from the set
copy()	Returns a copy of the set
difference()	Returns a set containing the difference between two or more sets
difference_update()	Removes the items in this set that are also included in another, specified set
discard()	Remove the specified item
intersection()	Returns a set, that is the intersection of two other sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)
isdisjoint()	Returns whether two sets have a intersection or not

issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set
remove()	Removes the specified element
symmetric_difference()	Returns a set with the symmetric differences of two sets
symmetric_difference_update()	inserts the symmetric differences from this set and another
union()	Return a set containing the union of sets
update()	Update the set with the union of this set and others

Input: (Write This)

Input 3 sets of students playing cricket, badminton and football.

Output: (Write This)

Union
A = { Amit, Ravi, Raj, Priya } B = { Ravi, Rahul, Raj, Meena}
A U B = { Amit, Ravi, Raj, Priya, Rahul, Meena }
Intersection
A = { Amit, Ravi, Raj, Priya } B = { Ravi, Rahul, Raj, Meena}
A c B = { Ravi, Raj}
Difference
A = { Amit, Ravi, Raj, Priya } B = { Ravi, Rahul, Raj, Meena}
A - B = { Amit, Priya}
Difference
A = { Amit, Ravi, Raj, Priya } B = { Ravi, Rahul, Raj, Meena}
B - A = {Rahul, Meena }

Theory Questions: (Write This)

1. What is array?
2. Explain Memory representation of array.
3. Explain List in python with example.
4. Explain Function in Python? (What is function, Declaration and definition of function, function arguments etc.)
5. Write a Python program to check if two given sets have no elements in common.

6. Given a Python list. Turn every item of a list into its square

Write algorithm/ pseudo code for following module:

1. Accept Student information/Sets
2. Display set
3. Finding union of two sets
4. Finding intersection of two sets
5. Finding difference of two sets

Sample PseudoCode

1. Here union of sets A and B will find out students who play either cricket or badminton or both.

```
Declare Union[n]        // n >= x + y
Copy the array A [ ] to Union [ ].
For every element x of array B [ ], do following
        Search x in array A[ ].
            If x  is not present, then copy it to Union[ ].
Return Union [ ].
```

2. Set of students who play both cricket and badminton

```
Declare Intersection [size].
Initialize array Intersection [ ] as empty.
For every element x of array A [ ], do following
      Search x in array B [ ].
          If x is present, then copy it to Intersection [ ].
Return Intersection [ ].
```

3. Set of students who play only cricket

```
Declare Array Only_cricket[ ];
Declare integer flag
For every element x of array A [ ], do following
        Search x in array B[ ].
    If x  is not present in B[ ] , then copy it to Only_cricket[ ]
Return Only_cricket[ ].
```

4. Set of students who play only badminton

```
Declare Array Only_Batminton[ ];
```

Declare integer flag
For every element x of array B[ ], do following
Search x in array A[ ].
If x is not present in A[ ] , then copy it to Only_Batminton[ ];
Return Only_Batminton[ ].

5. Number of students who play neither cricket nor badminton
   Here array *Students [ ]* represent set of all students in second year computer engineering.

Declare Array X[ ];
Declare integer flag
For every element x of array Students [ ], do following
Search x in array Union[ ].
If x is not present in Union[ ] , then copy it to X[ ]
Return X [ ].

Conclusion: (Write This)

Thus we have learnt to declare a 1D dimensional array, read the elements of the array and print the elements of the array. We have also implemented the following set operations on the arrays:
1. Union, 2. Intersection, 3. Difference

Viva Questions:
1. What is an array? What are its different types? Explain with example. Are there any drawbacks of an array? If so, explain.

2. What are the advantages and disadvantages of using arrays? Explain
3. Explain array as an ADT. Explain the representation of arrays in memory.
4. What is row major and column major storage representation methods of an array?
5. What do you mean by recursive function?
6 What are the commands to compile and execute a python program?
7. What are %s, %d with respect python?
8. What is the minimum index in an array?

Group: A **Roll No:**

Assignment No: 2 **Date:**

Title:

Write a python program to compute following operations on String:

a) To display word with the longest length
b) To determines the frequency of occurrence of particular character in the string
c) To check whether given string is palindrome or not
d) To display index of first appearance of the substring
e) To count the occurrences of each word in a given string

Marks: **Sign:**

Group A

Assignment No. A2

Title: String Operations (Write This)

Objectives: (Write This)

To implement different operations on strings like Creating, copying, modifying, concatenating, reversing , Finding substrings, etc.

To simulate the inbuilt string operations function.

Problem Statement: (Write This)

Write a python program to compute following operations on String:

a) To display word with the longest length
b) To determines the frequency of occurrence of particular character in the string
c) To check whether given string is palindrome or not
d) To display index of first appearance of the substring
e) To count the occurrences of each word in a given string

Input: (Write This)

Input a string. Ex. string1=" „

Output: (Write This)

1. To display word with the longest length
2. Frequency of occurrence of particular character in the string
3. String is palindrome or not
4. Index of first appearance of the substring
5. Occurrences of each word in a given string

Theory

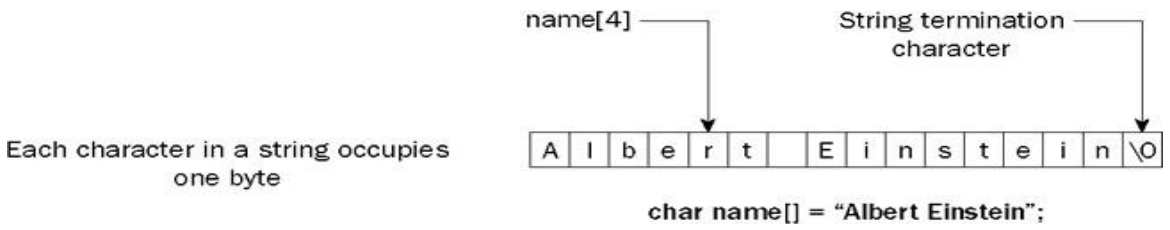
Explain string with example. (Write This)

Algorithm/ Pseudo code for following: **(Write This)**

1. Display word with the longest length
2. Determines the frequency of occurrence of particular character in the string
3. Check whether given string is palindrome or not
4. Display index of first appearance of the substring
5. Count the occurrences of each word in a given string

STRINGS

A string is an array of characters ended with NULL character („\0"). This null character indicates the end of the string. Strings are always enclosed by double quotes; whereas, character is enclosed by single quotes in C. Each character is stored in 1 byte as its ASCII code.



Substring : A string is a substring of a main string if it is a part of the main string.
Palindrome : A string is a palindrome if reverse of the string is equal to the original string.
e.g : nitin

Different string operations are as follows :

1. int strlen(const char *str) -- Calculates string length excluding last null character.
2. char * strcpy(char *dest, const char * src) – Copies string from source to destination
3. char *strcat(char *dest, const char * src) – Appends source string at the end of destination and returns pointer to destination
4. int strcmp(const char *str1, const char * str2) – Does unsigned comparison of two strings character by character and returns difference as integer.
5. String palindrome – Checking if reversed string is same as original string.

Conclusion: (Write This)

Thus the dtring have been implemented (without using standard library routines).

Oral Questions

Q1. What is a string? How do you know its length?
Q2. What are the main differences between a string constant and a character constant?
Q3. Does the gets() function save the newline character from the standard input stream?
Q4. What types of data can the scanf() function read?
Q5. What are the left and right values (lvalue & rvalue)?
Q6. How can you obtain the address of a variable?
Q7. What is the concept of indirection in terms of using pointers?
Q8. Can a null pointer point to valid data?
Q9. What is the main difference between using scanf & gets for accepting strings ?

Q10. How do you reference an array by using a pointer?

Group: A **Roll No:**

Assignment No: 3 **Date:**

Title:

Write a python program to compute following computation on matrix
a) Addition of matrices
b) Subtraction of matrices
c) Multiplication of matrices
d) Transpose of matrix

Marks: **Sign:**

Group A
Assignment No. 3 (Write)

Title: Matrix Operations (Write)

Objective: (Write)

- To introduce the basics of Matrices and some of its applications.
- To study the representation, implementation and applications of 2D Array data structures
- To understand the how to pass 2D arrays as parameters to functions.

Problem Statement (Write)

Write a python program to compute following computation on matrix:
1. Addition of matrices
2. Subtraction of matrices
3. Multiplication of matrices
4. Transpose of matrix

Problem Definition:

Matrix: A matrix is an m x n rectangular 2-D array of numbers with m rows and n columns. Or A matrix is an ordered set of numbers listed in rectangular form.

	Column 0	**Column 1**	**Column 2**	**Column 3**
row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Initializing Two-Dimensional arrays:

Two-Dimensional arrays may be initialized by providing a list of initial values & it looks like this:
int A[4][4] = { {0,1,2,3}, {3,2,1,0}, {3,5,6,1}, {3,8,3,4} };

We might write the two-dimensional array out as:

```
int A[4][4] =     { {0, 1, 2, 3},
                    {3, 2, 1, 0},
                    {3, 5, 6, 1},
                    {3, 8, 3, 4} };
```

For a two-dimensional array, in order to reference every element, we must use two nested loops.

Theory: (Write)

- 2-Dimentional Array
- Row major and column major storage representation for array
- Address calculation for two dimensional array

Algorithms: (Write)
Write down algorithms for following modules:

1. Accepting a Matrix
2. Displaying a Matrix
3. Finding Transpose of matrix
4. Adding two matrices
5. Subtraction of two matrices
6. Multiplication of two matrices

1. Addition of Two Matrices

If A and B are two matrices of same order m x n (to be read as m by n matrix) then their addition is defined by A + B. For example if A is a matrix of order 2×3 and B is a matrix of same order 2×3 then addition is possible and the resultant matrix will be of order 2×3. If a matrix A has its order 2×3 and another matrix B has its order 3×2 then addition is not possible. So before adding any number of matrices we must check that whether each of the matrices are of same order or not then we need to proceed further.

<u>Algorithm</u>

Suppose A and B are two matrix arrays of order m x n, and C is another matrix array to store the addition result. i, j are counters.

```
Step1: Start
Step2: Read: m and n
Step3: Read: Take inputs for Matrix A[1:m, 1:n] and Matrix B[1:m, 1:n]
Step4: Repeat for i := 1 to m by 1:
                Repeat for j := 1 to n by 1:
                        C[i, j] := A[i, j] + B[i, j]
                [End of inner for loop]
        [End of outer for loop]
Step5: Print: Matrix C
Step6: Exit.
```

2. Subtraction of Two Matrices

If A and B are two matrices having same order m x n and C be the resultant Matrix then we can write C = A – B. The order of Matrix C will also be m x n. To perform subtraction, order of each matrix should be same otherwise subtraction is not possible as it stated above in the case of Matrix Addition.

<u>Algorithm:</u>

Suppose A and B are two matrix arrays of order m x n, and C is another matrix array to store the addition result. i, j are counters.

```
1: Start
2: Read: m and n
3: Read: Take inputs for Matrix A[1:m, 1:n] and Matrix B[1:m, 1:n]
4: Repeat for i := 1 to m by 1:
        Repeat for j := 1 to n by 1:
                C[i, j] := A[i, j] – B[i, j]
        [End of inner for loop]
   [End of outer for loop]
5: Print: Matrix C[1:m, 1:n]
6: Exit.
```

3. Matrix Multiplication

If A and B are two matrices of order m x n and p x q then their multiplication possible only if the number of columns in first matrix is equal to the number of rows in second matrix. Suppose C a matrix where the result of A x B will be stored then the order of C is m x p.

<u>Algorithm</u>

1. Start
2. Declare variables and initialize necessary variables
3. Enter the element of matrices by row wise using loops
4. Check the number of rows and column of first and second matrices
5. If number of rows of first matrix is equal to the number of columns of second matrix, go to step 6. Otherwise, print matrix multiplications are not possible and go to step 3.
6. Multiply the matrices using nested loops.
7. Print the product in matrix form as console output.
8. Stop

<u>Pseudo Code</u>

Suppose A and B are two matrices and their order are respectively m x n and p x q. i, j and k are counters. And C to store result.

```
1. Start.
2. Read: m, n, p and q
3. Read: Inputs for Matrices A[1:m, 1:n] and B[1:p, 1:q].
4. If n ≠ p then:
        Print: Multiplication is not possible.
  Else
  {
        For( i =1 to n by 1)
        {
                For( j =1 to q bt 1)
                        C[i, j] := 0 [Initializing]
```

```
                    for( k=1 to n by 1)
                        C[i, j] := C[i, j] + A[i, k] x B[k, j]
            }
    }
5.  Display C
6.  Exit.
```

4. Transpose of A Matrix

A transpose of matrix can be formed by turning all the rows of a given matrix into columns and vice-versa. The transpose of matrix A is written A^T.

Example: $$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

First read a matrix of size mxn and then find its transpose by just interchanging the rows and columns i.e. rows become columns and columns become rows.

Algorithm:

Suppose A is a matrix array of order m x n. And B is a matrix array of order n x m to store result. i and j are two counters.

```
Step1: Start.
Step2: Read: m and n
Step3: Read: Take inputs for Matrix A[1:m, 1:n].
Step4: If m == n then:
            Repeat for i = 1 to m by 1
              Repeat for j = 1 to n by 1
                B[i, j] = A[j, i]
            [End of for loop]
          [End of for loop]
        Else:
              temp = m
              m = n
              n = temp
       Repeat for i = 1 to m by 1
         Repeat for j = 1 to n by 1
              B[i, j] = A[j, i]
        [End of for loop]
      [End of for loop]
      [End of If structure]
Step5: Print: B[1:m, 1:n]
Step6: Exit.
```

Sample Input Output:

1. Addition 2. Subtraction of matrices

$$A = \begin{bmatrix} -5 & 1 & -3 \\ 6 & 0 & 2 \\ 2 & 6 & 1 \end{bmatrix} \qquad B = \begin{bmatrix} 2 & 4 & 5 \\ -8 & 10 & 3 \\ -2 & -3 & -9 \end{bmatrix}$$

$$A + B = \begin{bmatrix} -3 & 5 & 2 \\ -2 & 10 & 5 \\ 0 & 3 & -8 \end{bmatrix} \qquad A - B = \begin{bmatrix} -7 & -3 & -8 \\ 14 & -10 & -1 \\ 4 & 9 & 10 \end{bmatrix}$$

Transpose of matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} = \begin{bmatrix} 1 & 5 & 9 \\ 2 & 6 & 10 \\ 3 & 7 & 11 \\ 4 & 8 & 12 \end{bmatrix}^{T}$$

Oral Questions:

1. How to allocate memory dynamically for a 2D-Array of order 4 x 5 ?
2. What is the difference between int *M[10] & int (*P)[10] ?
3. How do I know how many elements an array can hold?
4. How to allocate memory for a 3-D array of size 3 x 4 x 5 **.**
5. How does C compiler store elements in a multi-dimensional array?
6. How to find the row and column dimension of a given 2-D array?

Conclusion:

Thus we have learnt to declare a two dimensional array, read the elements of the array and print the elements of the array using static method.

1. Addition of matrices
2. Subtraction of matrices
3. Multiplication of matrices
4. Transpose of matrix

Design Experiments:

1. Write a python program to generate a Magic Square Matrix.

2. Write a python program to find out the inverse of an n x n matrix.

Group: B **Roll No:**

Assignment No: 4 **Date:**

Title:

a) Write a **python** program to store roll numbers of student in array who attended training program in random order. Write function for searching whether particular student attended training program or not, using Linear search and Sentinel search.

b) Write a **python** program to store roll numbers of student array who attended training program in sorted order. Write function for searching whether particular student attended training program or not, using Binary search and Fibonacci search

Marks: **Sign:**

Group B

Assignment No. 4

Title: Implement Searching algorithms(Write This)

Objectives: (Write This)

To understand and implement Linear search and Sentinel search

To understand and implement Binary search and Fibonacci search

Problem Statement: (Write This)

a) Write a **python** program to store roll numbers of student in array who attended training program in random order. Write function for searching whether particular student attended training program or not, using Linear search and Sentinel search.
b) Write a **python** program to store roll numbers of student array who attended training program in sorted order. Write function for searching whether particular student attended training program or not, using Binary search and Fibonacci search

Theory Questions: (Write This)

1) Write an Algorithm for Linear search with example. State its time and space complexity.
2) Write an Algorithm for Sentinel search with example. State its time and space complexity.
3) Write an Algorithm for recursive and non recursive binary search with example. State its time and space complexity.
4) Write an Algorithm for Fibonacci search with example. State its time and space complexity.
5) Compare Binary and Linear Search.
6) Compare Binary and Fibonacci Search.
7) How sentinel Search is better than Linear Search.

Group: B **Roll No:**

Assignment No: 5 **Date:**

Title:

Write a **python** program to store first year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using
a) Selection Sort
b) Bubble sort and display top five scores.

Marks: **Sign:**

Group B

Assignment No. 5

Title: Implement Sorting algorithms(Write This)

Objectives: (Write This)
To understand and implement selection Sort

To understand and implement Bubble Sort

Problem Statement: (Write This)

Write a **python** program to store first year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using
a) Selection Sort
b) Bubble sort and display top five scores.

Theory Questions: (Write This)

1) **What do you mean by internal and external sorting?**
2) **Write an algorithm for selection sort. Write space and best, average, worst time complexity.**
3) **Write an algorithm for bubble sort. Write space and best, average, worst time complexity.**
4) **Compare selection and Bubble sort.**
5) **Explain all steps of Selection sort using example.**
6) **Explain all steps of Bubble sort using example.**

Group B

Assignment No. 6

Title: Implement Insertion and Shell Sort algorithms(Write This)

Objectives: (Write This)
To understand and implement insertion Sort

To understand and implement Shell Sort

Problem Statement: (Write This)

Write a **python** program to store second year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using
a) Insertion sort
b) Shell Sort and display top five scores

Theory Questions: (Write This)

1) **Write an algorithm for Insertion sort. Write space and best, average, worst time complexity.**
2) **Write an algorithm for Shell sort. Write space and best, average, worst time complexity.**
3) **Compare Insertion and Shell sort.**
4) **Explain all steps of Insertion sort using example.**
5) **Explain all steps of Shell sort using example.**

Group: B **Roll No:**

Assignment No: 7 **Date:**

Title:

Write a **python** program to store first year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using quick sort and display top five scores.

Marks: **Sign:**

Group B

Assignment No. 7

Title: Implement Quick Sort algorithms(Write This)

Objectives: (Write This)
To understand and implement Quick Sort

To understand Time complexity of Quick sort

Problem Statement: (Write This)

Write a **python** program to store first year percentage of students in array. Write function for sorting array of floating point numbers in ascending order using quick sort and display top five scores.

Theory Questions: (Write This)

1) **Write an algorithm for Quick sort. Write space and best, average, worst time complexity.**
2) **Compare Quick sort and selection sort.**
3) **Explain all steps of quick sort using example.**
4) **Explain stable and unstable sort.**

Group: C **Roll No:**
Assignment No: 8 **Date:**

Title:
Write C++ program to implement Singly Linked List and operations on it-

a) Add and delete node
b) Compute total number of nodes
c) Traverse List
d) Display list in reverse order using recursion
e) Concatenate two lists

Marks: **Sign:**

Group C
Assignment No 8

Title: Implementation of singly linked list**(Write)**

Problem Statement(Write)

Department of Computer Engineering has student's club named 'Pinnacle Club'. Students of second, third and final year of department can be granted membership on request. Similarly one may cancel the membership of club. First node is reserved for president of club and last node is reserved for secretary of club. Write C++ program to maintain club member,,s information using singly linked list. Store student PRN and Name. Write functions to:

a) Add and delete the members as well as president or even secretary.
b) Compute total number of members of club
c) Display members
d) Two linked lists exists for two divisions. Concatenate two lists.

Theory:

Linked List is a linear data structure and it is very common data structure which consists of group of nodes in a sequence which is divided in two parts. Each node consists of its own data and the address of the next node and forms a chain.

fig. 1 Linked List

- **Singly Linked List:** Singly linked lists contain nodes which have a data part as well as an address part i.e. next, which points to the next node in sequence of nodes. The operations we can perform on singly linked lists are insertion, deletion and traversal.

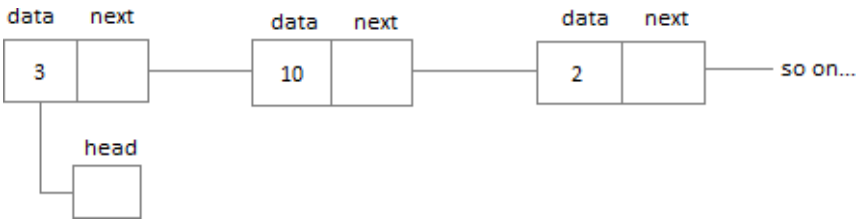


fig. 2 Singly Linked List

- What is a NODE?
  Node is a combination of DATA and LINK.
- What is Data?
  Is the part where the actual data is stored.
- What is a Link?
  It is the link (pointer) to the next element of the list.
- **Two ways of implementing link:**
  - It could be either an index to an array element (array implementation) OR

- A pointer variable containing the address of the next element (pointer)

Operations on linked list

- Creation
- Insertion
- Deletion
- Search
- Traverse

- **Operations on linked list:**

I. Insertion: A node can be added in four ways

- At the front of the SLL
- After a given node
- At the end of the SLL
- Before a given node

- **Add a node at the front:** The new node is always added before the head of the given Linked List. And newly added node becomes the new head of SLL.

  The following steps are to be followed:

  1. Create the new node.
     ```
     newnode=new Node(data);
     ```
  2. If the list is empty, assign new node as head
     ```
     head=newnode;
     ```
  3. If the list is not empty, follow the steps given below
     ```
     newnode->next=head;
     head=newnode;
     ```

- **Add a node at the end:** The new node is always added after the last node of the given Linked List.

  The following steps are to be followed:

  1. Create the new node.
     ```
     newnode=new Node(data);
     ```
  2. If the list is empty, assign new node as head
     ```
     head=newnode;
     ```
  3. If the list is not empty, follow the steps given below
     ```
     Node *temp=head;
     while(temp->next!=NULL)
               temp=temp->next;
            temp->next=newnode;
     ```

- **Add a node after a given node:** We are given pointer to a node as pre_node, and the new node is inserted after the given node.

  The following steps are to be followed:

  1. Create the new node.
     ```
     newnode=new Node(data);
     ```

2. Accept position.
   a. If position is not valid, display „**Invalid Position**" message.
3. If position is valid, follow the steps given below

```
Node *temp=head;
for(int i=1;i<=position;i++)
{
     prev=temp;
     temp=temp->next;
}
prev->next=newnode;
newnode->next=temp;
```

- **Add a node before a given node:** We are given pointer to a node as prev_node, and the new node is inserted after the given node.
  The following steps are to be followed:
  1. Create the new node.

```
     newnode=new Node(data);
```

  2. Accept position.
     a. If position is not valid, display „**Invalid Position**" message.
  3. If position is valid, follow the steps given below

```
Node *temp=head;
for(int i=1;i<position;i++)
{
     prev=temp;
     temp=temp->next;
}
     prev->next=newnode;
     newnode->next=temp;
```

II. Deletion: A node can be deleted in three way

- At the front of the SLL
- At the end of the SLL
- At the mid position

- **Deleting a node at the beginning of the SLL**
  The following steps are followed
  1. If the list is empty, display a message „Empty List".
  2. If the list is not empty, follow the steps given below:

```
cout<<"\nNode "<<head->data<<" deleted";
temp=head->next;
delete head;
head=temp;
```

- **Deleting a node at the end of the SLL**
  The following steps are followed
  1. If the list is empty, display a message „Empty List".

2. If the list is not empty, follow the steps given below:

```
temp=head;
while(temp->next!=NULL)
{
     prev=temp;
     temp=temp->next;
}
prev->next=NULL;
cout<<"\nNode "<<temp->data<<" deleted";
delete temp;
```

- **Deletion of a node at mid position**

  The following steps are to be followed:

1. Accept position.
   a. If position is not valid, display „Invalid Position‟ message.
2. If position is valid, follow the steps given below

```
while(counter<position)
{
     prev=temp;
     counter++;
     temp=temp->next;
}
prev->next=temp->next;
cout<<"\nNode "<<temp->data<<" deleted.";
delete temp;
```

III. Traversing SLL:

Following steps are followed, to traverse Singly Linked List.

1. If list is empty then display „Empty List‟ message.
2. If the list is not empty, follow the steps given below

```
temp=head;
while(temp!=NULL)
{
     cout<<"\n"<<temp->data;
     temp=temp->next;
}
```

IV. Concatenation of two Linked Lists:

1. Let us assume that the two linked lists are referenced by **head1** and **head2** respectively.
2. If the first linked list is empty then return **head2**.
3. If the second linked list is empty then return **head1**.
4. Store the address of the starting node of the first linked list in a pointer variable, say **temp**.

```
temp=head1;
```

5. Move the **temp** to the last node of the linked list through simple linked list traversal technique.

```
while(temp->next!=NULL)
   temp=temp->next;
```

6. Store the address of the first node of the second linked list in the next field of the node pointed by **temp**. Return **head1.**

```
temp->next=head2;
```

Conclusion(write)

Questions: (write)

1. Write C/C++ pseudo code to merge two singly linked list.

2. List and Explain types of linked list with example.

3. Write C/C++ psuedo code to search a node in Linked list.

4. Write C/C++ code to traverse list.

5. Write applications of linked list.

6. Write difference between Array and Linked List.

Group C **Roll No:**
Assignment No: 9 **Date:**

Title:
Write C++ program for storing binary number using doubly linked lists. Write functions

a) to compute 1,,s and 2,,s complement
b) add two binary numbers

Marks: **Sign:**

Group B

Assignment no 3

Title: (write)

Write C++ program for storing binary number using doubly linked lists. Write functions

a) to compute 1,,s and 2,,s complement

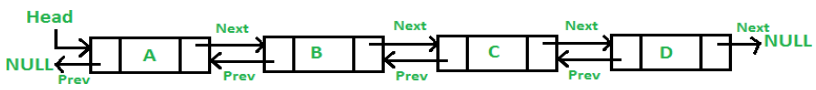
b) add two binary numbers

Index terms: Binary Numbers, Doubly linked list

Theory:

- **Doubly Linked list:**

A **D**oubly **L**inked **L**ist (DLL) contains an extra pointer, typically called *previous pointer*, together with next pointer and data which are there in singly linked list.



Node structure of DLL in C :

```
/* Node of a doubly linked list */
struct node
{
    int data;
    struct node *next; // Pointer to next node in DLL
    struct node *prev; // Pointer to previous node in DLL
};
```

• 1's and 2's complement of a Binary Number

1's complement of a binary number is another binary number obtained by toggling all bits in it, i.e., transforming the 0 bit to 1 and the 1 bit to 0.

Examples:

```
1's complement of "0111" is "1000"
1's complement of "1100" is  "0011"
```

2's complement of a binary number is 1 added to the 1"s complement of the binary number.
Examples:

```
2's complement of "0111" is  "1001"
2's complement of "1100" is  "0100"
```

Algorithm:

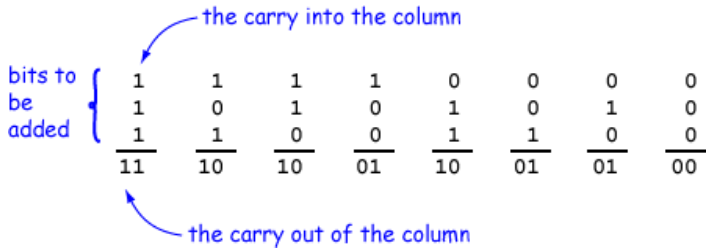
1. Read decimal value from user.
2. Convert it into binary format.
3. Store all digits in Doubly linked list.
4. Finds the 1's complement of the binary number

```
for(i=0; i<SIZE; i++)
{
  if(binary[i]=='1')
  {  onesComp[i] = '0';
  }  else if(binary[i]=='0')
  {  onesComp[i] = '1';
  }
}
onesComp[SIZE] = '\0';
```

5. Find 2‟ s complement by adding one to 1's complement.
6. Print 1's complement and 2s complement.

- ## Addition of binary numbers:

Rules for binary addition:



Conclusion(write)

Questions(write)

1. Write a C/C++ program to convert decimal number to binary number.
2. Write a C/C++ program for multiplication of binary number.

3. Write advantages of DLL over singly linked list.
4. Write a C/C++ program for subtraction of binary number

Group D **Roll No.**
Assignment No: 10 **Date:**

Title: Stack using Arrays

Marks: **Sign:**

Group: D

Assignment No. 10

Title(write) Stack Using Arrays

Problem Statement(Write)

A palindrome is a string of character that,,s the same forward and backward. Typically, punctuation, capitalization, and spaces are ignored. For example, “Poor Dan is in a droop” is a palindrome, as can be seen by examining the characters “poor danisina droop” and observing that they are the same forward and backward. One way to check for a palindrome is to reverse the characters in the string and then compare with them the original-in a palindrome, the sequence will be identical. Write C++ program with functions-
a) To print original string followed by reversed string using stack
b) To check whether given string is palindrome or not

Index terms: Stack, PUSH(), POP()

Theory:

Stack:

A stack is a container of objects that are inserted and removed according to the last-in first-out (LIFO) principle. In the pushdown stacks only two operations are allowed: **push** the item into the stack, and **pop** the item out of the stack. A stack is a limited access data structure - elements can be added and removed from the stack only at the top. **Push** adds an item to the top of the stack, **pop** removes the item from the top. A helpful analogy is to think of a stack of books; you can remove only the top book, also you can add a new book on the top.

A stack is a **recursive** data structure. Here is a structural definition of a Stack:

a stack is either empty or it consists of a top and the rest which is a stack;

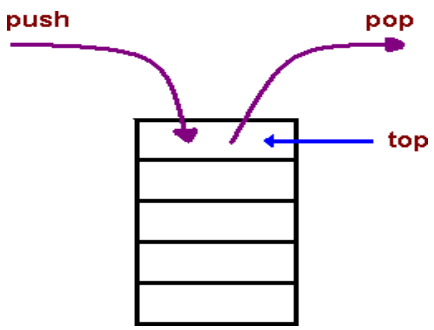


Fig. Stack

Operations on Stack:

1. PUSH()

2. POP()

3. IsEmpty()

4. IsFull()

Algorithm:

1. **To check string is palindrome or not**

- Find the length of the input string using strlen function and store it in a integer variable "length".
- Using a for loop, traverse input string from index 0 to length-1 and push all characters in stack.
- Remove (Pop) characters from stack one by one using a for loop and compare it with corresponding character of input string from beginning(traverse from index 0 to length-1). If we found a mismatch the input string is not a palindrome string otherwise **palindrome string**.

2. **Reverse a string**

- 1 Find the length of the input string using strlen function and store it in a integer variable "length".
- Using a for loop, traverse input string from index 0 to length-1 and push all characters in stack.
- Remove (Pop) characters from stack one by one using a for loop.
- Print the string

Conclusion: (write)

Questions: (write)

1. **What is stack? Write an ADT for stack.**
2. **What is recursion? Explain use of stack for recursion.**
3. **Write applications of stack.**
4. **Represent stack using an array**
5. **Represent stack using linked list**
6. **Write a program to reverse a number using stack.**
7. **Explain the Push operations?**
8. **Explain the Pop Operations?**
9. **Explain the stack empty condition?**
10. **Explain the stack full condition?**

Group: D **Roll No:**

Assignment No: 11 **Date:**

Title : Expression conversion using stacks

Marks: **Sign:**

Group D

Assignment No:11

Title: **Expression conversion using stacks** (Write This)

Objectives: (Write This)

- To implement stack as an ADT using templates
- To understand implement the expression conversions methods.

Problem Statement: (Write This)

Implement C++ program for expression conversion as infix to postfix and its evaluation using stack based on given conditions

1. Operands and operator, both must be single character.
2. Input Postfix expression must be in a desired format.
3. Only '+', '-', '*' and '/ ' operators are expected

Theory: (Write This)

1. Expression (Infix, prefix and postfix notation)
2. Advantages of Postfix and Prefix expressions

Write algorithms for following modules: (Write This)

1. Algorithm for Infix to Postfix Conversion
2. Algorithm for Postfix Evaluation

Conclusion: (Write This)

Thus we have implemented stack as an abstract data type (ADT) and applied it for Expression conversion and evaluation.

Oral Questions:

What's the idea behind templates? What's the syntax / semantics for a "class template" & "function template"?

Q2. Explain Postfix notation?

Q3. Explain Prefix notation?

Q4. Why would anyone ever want to use anything so “unnatural” (postfix / Prefix) when infix seems to work just fine?

Q5. Convert the expression ((A + B) * C - (D - E) ^ (F + G)) to equivalent Prefix and Postfix notations?

Q6. Convert the expression (((A/(B^C))+(D&E))-(A&C)) to equivalent Prefix and Postfix notations?

Q7. Explain what is recursion?

Q8. List the different types of recursion?

Q9. Explain Tail recursion?

Q10. Explain Implicit & Explicit Stack?

Design Experiments:

1. Write a C++ program to implement stack as an ADT using CDLL Linked list and use this ADT for following conversions

   i. Infix to Prefix conversion ii. Prefix evaluation

2. Write a C++ program to implement stack as an ADT using DLL Linked list and use this ADT for following conversions

   i. Prefix to Infix conversion ii. Prefix to Postfix conversion

FOR YOUR REFERENCE

Expressions : An algebraic expression is a legal combination of operands and the operators. Operand is the quantity (unit of data) on which a mathematical operation is performed. Operand may be a variable like x, y, z or a constant like 5, 4,0,9,1 etc. Operator is a symbol which signifies a mathematical or logical operation between the operands. Example of familiar operators include +,-,*, /, ^ .Considering these definitions of operands and operators now we can write an example of expression as x+y*z.

Infix notation : From our schools times we have been familiar with the expressions in which operands surround the operator, e.g. x+y, 6*3 etc this way of writing the Expressions is called infix notation. We usually write algebraic expressions like this: a + b. This is called infix notation, because the operator (“+”) is inside the expression. A problem is that we need parentheses or precedence rules to handle more complicated expressions:

For Example : a + b * c = (a + b) * c ?

= a + (b * c) ?

Prefix notation was introduced by the Polish logician Lukasiewicz, and is sometimes called "Polish notation". In the prefix notation, operator comes before the operands, e.g. +xy, *+xyz etc.

Postfix notation is sometimes called "reverse Polish notation" or RPN. They are different from the infix and prefix notations in the sense that in the postfix notation, operator comes after the operands, e.g. xy+, xyz+* etc.

Advantages of Postfix and Prefix expressions

1. Infix Expression Is Hard To Parse and Need operator priorities, tie breaker, and delimiters. This makes computer evaluation more difficult than is necessary.
2. Postfix and prefix expression forms do not rely on operator priorities, a tie breaker, or delimiters.
3. So it is easier to evaluate expressions that are in these forms.

Algorithm for Infix to Postfix Conversion

Algorithm Convert_infix_to_postfix(infix expression)

1 Start
2 Examine the next element in the input / infix expression.
3 If it is operand, output it.
4 If it is a closing parenthesis, pop operators from stack and output them until an opening parenthesis is encountered. pop and discard the opening parenthesis.
5 If it is opening parenthesis, push it on stack.
6 If it is an operator, then
    i) If stack is empty, push operator on stack.
    ii) If the top of stack is opening parenthesis, push operator on stack
    iii) If it has higher priority than the top of stack, push operator on stack.
    iv) Else pop the operator from the stack and output it, repeat step 6
7 If there is more input go to step 1
8 If there is no more input, pop the remaining operators to output.
9 Stop

Algorithm for Postfix Evaluation

```
Algorithm evaluate_Postfix_expression(Postfix expression)
WHILE more input items exists in postfix expression
{
        If symb is an operand
                then push (opndstk,symb)
        else //symbol is an operator
        {
                Opnd2=pop(opndstk);
                Opnd1=pop(opndnstk);
                Value = result of applying symb to opnd1 & opnd2
                 Push(opndstk,value);
```

```
        } //End of else
} // end while
Result = pop (opndstk);
```

Group: E **Roll No:**

Assignment No: 12 **Date:**

Title: Linear Queue

Marks: **Sign:**

Group E
Assignment No:12

Title: Linear Queue (Write This)

Objectives: (Write This)

- To understand the concept of linear Queue
- To implement Linear Queue as an ADT

Problem Statement: (Write This)

Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system. If the operating system does not use priorities, then the jobs are processed in the order they enter the system. Write C++ program for simulating job queue. Write functions to add job and delete job from queue.

Theory: (Write This)

1. Queue (definition ,example)
2. Queue ADT
3. Basic operations on Queues
4. Applications of Queues

Write algorithms for: (Write This)

3. Inserting an element into Linear Queue
4. Deleting an element from Linear Queue
5. Checking underflow
6. Checking Overflow

Conclusion: (Write This)

Thus we implement the Queue data structure and applied it for job scheduling problem.

Oral Questions:

Q1. Define Linear Queue?

Q2. What are the disadvantage of Linear Queue?

Q3. How the overcome these disadvantage of linear queue?

Q4. Explain few applications of Queue?

Q5. A linear queue using array has a size of 3. perform the following operations on this queue and show the sequence of steps with necessary diagrams indicating the values of front, rear and contents :

a. Insert 10 b. Insert 20
c. Insert 30 d. Delete an element
e. Insert 40 f. Delete an element
g.Delete an element h. Delete an element
i. Insert 50

Q6. Suggest the appropriate data structure for the following application :

a .Online air line reservation system.
b. Scheduling of jobs based on their target dates
c. To check for well formed parenthesis

Q7. How is multiple queue implemented using single large array.

Q8. What is queue simulation ? Explain how queue is simulated

Q9. Give the application of an double ended queue?

Q10. Explain the job scheduling application of Queue?

Design Experiments:

1. Write an C++ Program to implement multiple queue using single large array.
2. Write an C++ Program to keep track of the online air line reservation system.
3. Write an C++ Program to implement levelwise traversal for a tree.

FOR YOUR REFERENCE

Queue : A queue is a data structure that models/enforces the first-come first-serve order, or equivalently the first-in first-out (FIFO) order. A queue is a particular kind of collection in which the entities in the collection are kept in order and the principal (or only) operations on the collection are the addition of entities to the rear terminal position and removal of entities from the front terminal position. This makes the queue a First-In-First-Out (FIFO) data structure. In a FIFO data structure, the first element added to the queue will be the first one to be removed. This is equivalent to the requirement that once an element is added, all elements that were added before have to be removed before the new element can be invoked. A queue is an example of a linear data structure.

Abstract data type (ADT) : An Abstract Data type is defined as a mathematical model of the data objects that make up a data type as well as the functions that operate on these objects. There

are no standard conventions for defining them. A broad division may be drawn between "imperative" and "functional" definition styles. In general terms, an abstract data type is a *specification* of the values and the operations that has two properties:
□ it specifies everything you need to know in order to use the data type
□ it makes absolutely no reference to the manner in which the data type will be implemented.

When we use abstract data types, our programs divide into two pieces:
The Application: The part that uses the abstract data type.

Queue as an ADT :
AbstractDataType queue
{
instances
ordered list of elements; one end is the front; the other is the rear;
operations
isEmpty(): Return true if queue is empty, return false otherwise
isFull() : Return true if queue is full, return false otherwise
get_Size(): Return the number of elements in the queue
get_front(): Return the front element of queue
get_rear(): Return the back (rear) element of queue
dequeue(): Remove an element from the queue
enqueue(x): Add element x to the queue
}

Basic operations on Queues :
- Enqueue (item): (also called Insert): It adds a new item to the tail of the queue
- Dequeue ( ): (also called delete or Remove): It deletes the head item of the queue, and returns to the caller. If the queue is already empty, this operation returns NULL
- getHead( ): Returns the value in the head element of the queue
- getTail( ): Returns the value in the tail element of the queue
- isEmpty( ) : Returns true if the queue has no items
- isFull( ) : Returns true if the queue queue is full
- size( ) : Returns the number of items in the queue

Applications of Queues :
- An electronic mailbox is a queue
  The ordering is chronological (by arrival time)
- A waiting line in a store, at a service counter, on a one-lane road
- Equal-priority processes waiting to run on a processor in a computer system
- Printer queue,
- keystroke queue
- PRACTICAL EXAMPLE : A line at a ticket counter for buying tickets operates on above rules
- IN COMPUTER WORLD : In a batch processing system, jobs are queued up for processing.

Algorithms :
Algorithm for inserting an element into Linear Queue
Algorithm Enqueue (X)
1. Start
2. Increment rear by 1.

3. Place the record X at rear position i.e Q[rear] = X;

4. Stop

Algorithm for deleting an element from Linear Queue
Algorithm Dequeue ( )
1 Start

2 Assign the front record to X i.e X = Q[front]

3 Increment front by 1

4 Return X

5 Stop

Algorithm for Checking underflow
Algorithm isQEmpty()
1 Start

2 If front > rear return Q is Empty

3 Else return Q is not Empty

4 Stop

Algorithm for Checking Overflow
Algorithm isQEmpty()
1 Start

2 If rear == MAX - 1 return Q is FULL

3 Else return Q is not FULL

4 Stop

Group: E **Roll No:**

Assignment No: 13 **Date:**

Title: Double Ended Queue (Deque)

Marks: **Sign:**

Group E

Assignment No:13

Title: **Double Ended Queue (Deque)** (Write This)

Objectives: (Write This)

- To understand the concept of Deque
- To implement deque as an ADT

Problem Statement: (Write This)

A double-ended queue (deque) is a linear list in which addition and deletion may be made at either end. Obtain a data representation mapping a deque into a one-dimensional array.
Write a c++ program to simulate deque with functions to add and delete elements from either end of deque.

Theory: (Write This)

1. Double ended queue (definition, example, Types of deque)
2. Basic operations on Deque
4. Applications of deque

Write algorithms for: (Write This)

7. Inserting an element from front and rear
8. Deleting an element from front and rear
9. Checking underflow
10. Checking Overflow

Conclusion: (Write This)

Thus we implement the deque data structure.

Oral Questions:

Q1. Define Deque?

Q2. What is queue simulation? Explain how queue is simulated

Q3. Give the application of double ended queue?

Design Experiments:

1. Write an C++ Program to implement input restricted deque.

2 Write an C++ Program to implement Output restricted deque.

FOR YOUR REFERENCE

Deque or Double Ended Queue is a type of queue in which insertion and removal of elements can be performed from either from the front or rear. Thus, it does not follow FIFO rule (First In First Out).

Types of Deque

Input Restricted Deque

In this deque, input is restricted at a single end but allows deletion at both the ends. (draw diagram)

Output Restricted Deque

In this deque, output is restricted at a single end but allows insertion at both the ends. (draw diagram)

Applications of Deque Data Structure

In undo operations on software.

To store history in browsers.

For implementing both stacks and queues.

Group: E **Roll No:**

Assignment No: 14 **Date:**

Title: Circular Queue

Marks: **Sign:**

Group E

Assignment No:14

Title: Circular Queue (Write This)

Objectives: (Write This)

- To understand the concept of Circular Queue.
- To implement the Circular queue data structure.

Problem Statement: (Write This)

Pizza parlour is accepting maximum M orders. Orders are served in first come first served basis. Order once placed cannot be cancelled. Write C++ program to simulate the system using circular queue using array.

Theory: (Write This)

1. Circular Queue (definition ,example)
2. Advantage of circular queue
3. Circular queue using array

Write algorithms for: (Write This)

11. Inserting an element into circular Queue (Enqueue)
12. Deleting an element from circular Queue (Dequeue)
13. Checking underflow
14. Checking Overflow

Conclusion: (Write This)

Thus we have implemented Circular queue as an abstract data type (ADT) using arrays.

Oral Questions:

Q1. What is a circular queue? Why is it better than a normal queue?
Q2. Give some practical Examples of usage of circular queue?
Q3. List the disadvantages of linear queues?
Q4. Explain the concept of multiqueues?
Q5. Explain the concept of Dequeues?
Q6. What is the application of a circular queue?
Q7. Find the Time Complexity for queue operations using Array and linklist based Implementations?
Q8. Explain Job Scheduling application of queue?
Q9. What is the difference between a queue and an Array?
Q10. Write a formula to find number of elements in a Circular Queue i.e. size of queue?

Design Experiments:

1. Write a C++ program to implement Circular queue as an ADT using SLL Linked list
2. Write a C++ program to implement Round robin job scheduling algorithm using circular queues.
3. Write a C++ program to find the shortest path between grid positions a and b in Wire Routing problem using queues.

FOR YOUR REFERENCE

Circular Queue

When a new item is inserted at the rear, the pointer to rear moves upwards. Similarly, when an item is deleted from the queue the front arrow moves downwards. After a few insert and delete operations the rear might reach the end of the queue and no more items can be inserted although the items from the front of the queue have been deleted and there is space in the queue. To solve this problem, queues implement wrapping around. Such queues are called Circular Queues. Both the front and the rear pointers wrap around to the beginning of the array. It is also called as “Ring buffer”. Items can inserted and deleted from a queue in O(1) time.

Algorithms :
Algorithm for inserting an record into Circular Queue
Algorithm Enqueue (X) where X is the record to be added in the Circular Queue

1. Start
2. Increment rear logically using the formula rear = (rear + 1) % MAX where MAX is the maximum size of queue.
3. Place the record X at rear position i.e CQ[rear] = X;
4. Stop

Algorithm for deleting an record from Circular Queue

Algorithm Dequeue ( )

1 Start

2 Assign the front record to X i.e X = CQ[front]

3 If the record deleted happens to be last record in the CQ then reinitialize front = 0 and rear = -1

4 Else Increment front logically using the formula front = (front+1)%MAX

5 Return the record X

6 Stop

Algorithm for Checking underflow

Algorithm isQEmpty()

1. Start
2. If rear == -1 return CQ is Empty
3. Else return CQ is not Empty
4. Stop

Algorithm for Checking Overflow

Algorithm isQEmpty()

1. Start
2. If (rear + 1) % MAX == front and rear != -1 return CQ is FULL
3. Else return CQ is not FULL
4. Stop